

ELEC6234 – FPGA Synthesis of a picoMIPS

Peter Xuanyuan Chai
Xc5g15
Meng Electronic Engineering with AI
Professor Rob Maunder

ABSTRACT: *A custom implementation of the picoMIPS processor was implemented onto an Altera DE1-SoC FPGA using System Verilog HDL. The template from the course was adapted such that the processor would be as small as possible. On testing all of the individual components on Modelsim, the design was synthesised with the use of 66 ALMs in total, resulting in a cost function of 71.07.*

1 Introduction

The objectives of this assignment is to design a design, synthesise and test a small picoMIPS processor in System Verilog implemented on an Altera DE1-SoC FPGA Development kit. The specification for the design is as follows:

1. It must be able to execute the affine transformation algorithm as provided in the brief. Where the inputs are given from the switches on the FPGA, with another switch acting to stage and write the contents of the switches.
2. The results must be displayed on the LEDs of the FPGA
3. It must not be dedicated hardware and the design must operate in a fashion akin to a processor.
4. It should be as small as possible, bearing in mind the cost function.

An overall system architecture was designed before starting any development, and this was to ensure a cohesive processor system could be referred to when writing HDL code. The component parts of the processor was divided into essentially two roles: Control Path and Data Path. The top-level is defined in the file “cpu.sv” and this is wrapped under the “picoMIPStest.sv” file provided. The top-level included all of the necessary components, as well as the necessary control and data wires for connection between the modules.

Each component was test benched on ModelSim to ensure functionality before system integration, which was also test benched with the “affine2.hex” source file. Once that was complete, Quartus 16.1 was used to synthesise and upload the source code onto the FPGA with the same hex file as in simulation

2 Affine Transformation Algorithm

It was required to implement the affine transformation algorithm as below, and the data set I have chosen to use is next to it.

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = A \times \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} + B \quad A = \begin{bmatrix} 0.75 & 0.5 \\ -0.5 & 0.75 \end{bmatrix} \quad B = \begin{bmatrix} 20 \\ -20 \end{bmatrix}$$

Bit position	Weight
7	-2^0
6	2^{-1}
5	2^{-2}
4	2^{-3}
3	2^{-4}
2	2^{-5}
1	2^{-6}
0 (LSB)	2^{-7}

The X_1 and Y_1 values are the inputs to the algorithm, and the outputs are X_2 and Y_2 . In order for the CPU to be able to determine the fractions in the A matrix, the fixed point representation on the left was used. In the ALU, 2's complement numbers would be multiplied as they normally would be a portion of the result would be taken to accurately represent the result from multiplying with a fraction. This will be explained further in the ALU section.

3 Control Path

The control path includes 4 modules in total: decoder, ROM (program memory), program counter, and the port module. The overall system architecture is provided in the block diagram below.

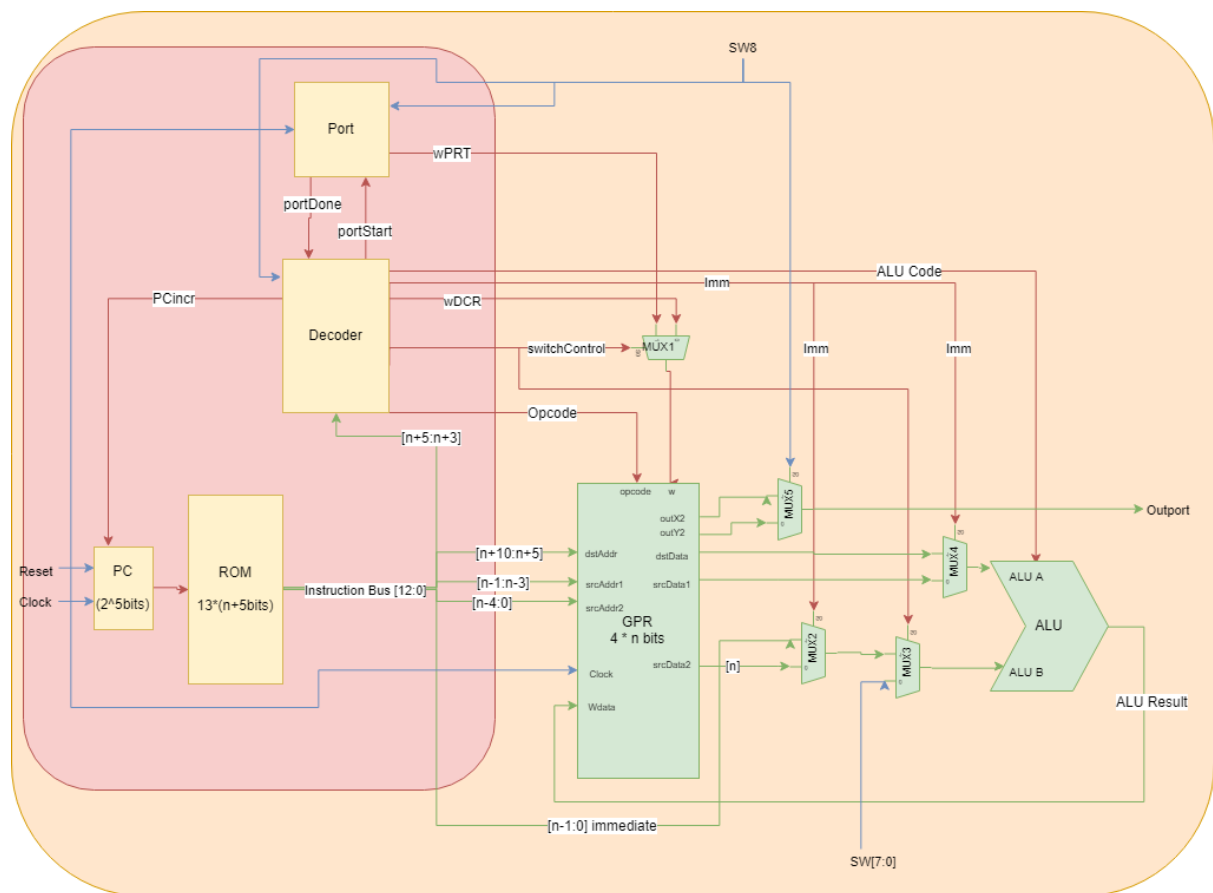


Figure 1. System Architecture diagram. Red is the region of the Control Path, the rest is the Data Path, with modules in Green. Red lines are control signals, Green lines are data signals, blue lines are external signals.

The Control Path is comprised of elements which provide control signals, with the exception of the ROM. The remainder of the architecture describes how the General Purpose Register (GPR) is read and written to using said control signals, and there are 5 multiplexers.

1. MUX1 determines whether the Port or Decoder module has the control over the write signal to the GPR.
2. MUX2 determines whether to take the immediate literal from the instruction bus depending on if the instruction is determined to be an immediate by the decoder.
3. MUX3 determines whether the decoder has decided to connect the B input to the ALU to the switches or the output of MUX2.
4. MUX 4 determines whether to connect the destination data of the register to the ALU, depending on if the instruction is immediate. If not the source 1 register will be connected.
5. MUX 5 determines which output register to connect to the output, depending on the condition of SW8.

3.1 Instruction Format

The instruction format or Instruction Set Architecture (ISA) is custom designed for the affine transformation algorithm. The instruction size is 13 bits with the opcode taking 3 bits. The type of ISA was decided to be GPR based [1], given that the task is to make a picoMIPS. There are 3 instruction types used, 2 of which are inspired by the R and I type instructions from the MIPS architecture. There is also a specialised instruction type for instruction LDP. Table 1 shows the three instruction formats.

Type	MSB <- Format in bits -> LSB												
	12	11	10	9	8	7	6	5	4	3	2	1	0
R	opcode			dst address		X	X	src1 address		X	X	src2 address	
I	opcode			dst address		immediate literal							
P	opcode			dst address		X	X	X	X	X	X	X	X

Table 1. Instruction format of R, I and P type instructions. "dst" stands for destination, "src" stands for source "X" means those values are ignored.

In the spirit of keeping the processor small, the least GPRs were used, thus only 2 bits of address was used because the minimal design only required 4 general purpose registers. However, if necessary, the source files could easily be changed to accommodate for 4 bit address sizes, allowing for up to 16 GPR to be addressed. No more than 4 bit address sizes should be used to keep the format efficient such that src1 and src2 addresses combined has the same size as the immediate literal, which must be 2's compliment 8 bit for the affine transformation algorithm.

3.2 Decoder

Table 2 shows the ISA and the 6 opcodes implemented for the decoder, and there is an I equivalent to each R instruction, as well as a specific instruction to perform loading values from the switches. These instructions were encoded with the pre-processor instruction `DEFINE.

OPCODE	NAME	DESCRIPTION
LDI	Load Immediate	Load a string literal into dst
LDP	Load Port	Pause the PC, and initiate the port load procedure, then un-pause the PC once finished.
ADD	Add	Add the data from src1 and src2 and store in dst.
ADDI	Add Immediate	Add the immediate literal to the dst data, and store in dst.
MUL	Multiply	Multiply the data from src1 and src2 and store in dst.
MULI	Multiply Immediate	Multiply the immediate literal to the dst data, and store in dst.

Table 2. ISA of CPU with the necessary functions to execute the affine transformation algorithm. “dst” refers to destination register, “src” refers to source register.

You will notice that there are no subtract or branch instructions as in the template. This is because in my design, these were unnecessary and were dead weight in terms of hardware, and so they were removed accordingly. The only difference is the LDP instruction which requests a load procedure from the Port module. It also acts as a way of displaying the data stored in the result registers, and this is handled by the Port module. The logic behind using a separate port module to handle taking the inputs from the switches is primarily that the decoder will be isolated from having to do anything that wouldn’t traditionally be the decoder’s job. Of course, there are other ways of implementing switch inputs, but I felt that having a Port plugin with a specific purpose makes the CPU more modular, akin to I/O in modern day computers.

In theory, it is still possible to perform the affine transformation without the immediate ADDI and MULI instructions, but this would require the use of many more GPRs, which are quite costly in terms of logic usage, and thus it was determined that it was worth having immediate functionality.

The two inputs, apart from the opcode, are “loadSwitch” which attached to SW8, and “portDone” which informs the decoder that the Port module has done the loading and the program counter can continue.

SIGNAL	ASSOCIATED OPCODE	DESCRIPTION
PCincr	LDP	Increment the program counter if high.
ALUfunc	All opcodes	Select the ALU function from the set of functions.
imm	LDI, ADDI, MULI	Immediate signal to use 8-bit immediate literal to ALUB instead of src2 data.
switchControl	LDP	Controls external multiplexers to hand over write control to the port and ALUB to the switches.
portStart	LDP	Start the load port procedure to load switch values into registers.
w	All opcodes	Controls the registers write function

Table 3. Decoder output control signals.

The decoder was implemented as a combinational block with a case statement for each of the opcode inputs. There is also a number of output control signals that the decoder controls, and these are explained in table 3. Although the ‘w’ signal deprecated since all opcodes make it high, it was kept to allow functions like branch or jumps in the CPU philosophy, and it doesn’t add much logic anyways.

Figure 2 shows the functionality of the control signals, and each of the opcodes were stepped through. For opcode LDP, a two way handshake protocol was used, and once the “portStart” signal is given by the decoder, it will wait with the “PCIncr” (so it will stay on the same instruction) low until the Port module returns a “portDone” signal. The instruction for LDP will also include a destination address, so the switch data can be written to a particular register. MUX1 and MUX2 will also select the control signals given by the Port module instead of the decoder if the load procedure hasn’t finished.

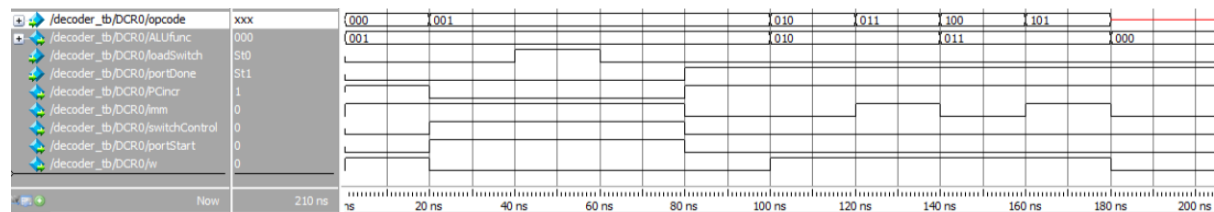


Figure 2. Testbench for decoder showing the control signals operation for each of the opcodes. The red signal shows unknown opcode and the control signals are set to the default values.

Default values for each output control signal were also set before the case statements so that only the signals that needed to be changed were affected. There is also a default case, where the decoder sees an unimplemented opcode, and this will throw an error.

3.3 Program Counter

This module is very simple as there is no branch functionality included in this program counter. The only control signal necessary is the “PCIncr” signal, which controls whether the PC will increment on a positive edge of the clock or pause. On asynchronous reset, the program counter will be hard set to 0. There is also a parameter for setting the size of the program counter register, this was set to be 5 bits, giving enough room to reference the entire ROM and more in the case that I decide to increase the size of the ROM.

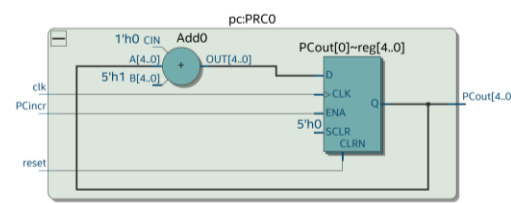


Figure 3. RTL diagram of the PC, a 5 bit adder and 5 simple flip-flops with the PCIncr signal as the enable, and the reset as a clear.

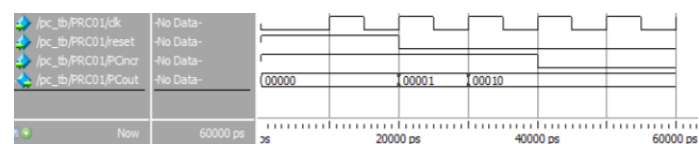


Figure 4. Modelsim simulation demonstrating the functionality of the PC, with all signals working correctly. PCout only increments when reset is low and when PCIncr is high.

3.4 ROM

The ROM, also known as the program memory, contains the program the CPU is to execute. This is loaded in from a hex file at compilation, and is a read only block of memory (synchronous RAM) as the module does not have write functionality. The hex file contains the instructions for that will execute the affine transformation algorithm, and it is written to the memory block by the \$readmemh() function.

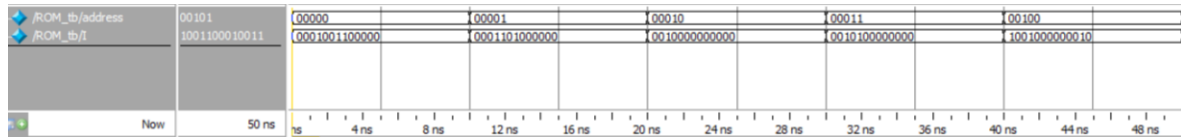


Figure 5. Simulation of the ROM returning the instruction stored when queried with a memory address.

Since there are, in total, 13 instructions for the program, the memory block has 13 instructions, which each instruction 13 bits in length as in the ISA. This means the entire size of the ROM is 169 bits.

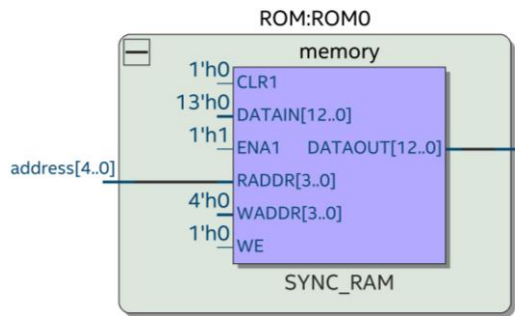


Figure 6. RTL diagram of the ROM, note that the write enable and write address pins are tied low, meaning the RAM is read only.

```

1 // Affine Transformation picoMIPS program
2 // n = 8 bits, Isize = 5+n = 13 bits
3 // format: 3b opcode, 4b %dst, 4b %src1 + 4b %src2 OR 8b immediate
4 //
5 // HEX ////////// BINARY ////////// ASSEMBLER //////////
6 0260 // 13'b000_10_0110_0000 LDI (Load 0.75)
7 0340 // 13'b000_11_0100_0000 LDI (Load 0.5)
8 0400 // 13'b001_00_0000_0000 LDP into gpr[0]
9 0500 // 13'b001_01_0000_0000 LDP into gpr[1]
10 1202 // 13'b100_10_0000_0010 MUL (0.75 a11*x1)
11 1313 // 13'b100_11_0001_0011 MUL (0.5 a12*y1)
12 0A32 // 13'b010_10_0011_0010 ADD (a11*x1 + a12*y1)
13 0E14 // 13'b011_10_0001_0100 ADDI (a11*x1 + a12*y1 + B) ANS
14 14C0 // 13'b101_00_1100_0000 MULI (-0.5 a21*x1)
15 1560 // 13'b101_01_0110_0000 MULI (0.75 a22*y1)
16 0B01 // 13'b010_11_0000_0001 ADD (a21*x1 + a22*y1)
17 0FEC // 13'b011_11_1110_1100 ADDI (a21*x1 + a22*y1 + B) ANS
18 0500 // 13'b001_01_0000_0000 LDP into gpr[1] to display

```

Listing 1. “affine2.hex” Hex code for the affine transformation. Instructions which produce and store the X2 and Y2 results are labelled with “ANS”.

You can see in listing 1 that the necessary numbers are loaded into all the 4 registers before calculating on them, and with the successive order of instructions it will be able to calculate X2 and Y2 according to the affine transformation algorithm.

3.5 Port

The Port module serves to:

1. Evaluate the condition of SW8 and the order in which it switches.
 - a. Low to high writes the status of SW[7:0] to ALUB, thus writing to the destination register.
 - b. High to low signals that the Port has served its purpose, thus setting the “portDone” signal high.

```

if (portStart == 0) begin
    w ≤ 0;
    portDone ≤ 0;
    lastState ≤ thisState;
end else begin

```

Listing 2. At the start of the always_ff block, there is a synchronous reset signalled by portStart, and if the decoder has not given this signal, then the Port module will not start its function. The state is still being tracked.

There are two 1 bit logic elements known as “thisState” and “lastState,” where “thisState” is tied to the “loadSwitch” input signal which is SW8. These are used to determine whether the transition of the switch is from low to high, or high to low. A synchronous always_ff block was used to implement this Port module, and the natural weakness of this is that if the switch goes from high then back to low (or vice-versa) too quickly, it may be possible that the change was not registered because the change occurred before the next clock edge. This is especially true if the clock used is slow. It may

have been wise to implement a separate faster clock for this module, but the performance was tested on the FPGA and was deemed adequate.

As in listing 2, the module waits for the “portStart” signal from the decoder. Listing 3 shows the code for the state machine, and as a default case, the state will be just tracked.

```
case(thisState)

0: begin

    if ( thisState = lastState ) begin
        lastState ≤ thisState;
    end else begin
        portDone ≤ 1;
        w ≤ 0;
        lastState ≤ thisState;
    end
end

1: begin

    if ( thisState = lastState ) begin
        lastState ≤ thisState;
    end else begin
        portDone ≤ 0;
        w ≤ 1;
        lastState ≤ thisState;
    end
end

default:    lastState ≤ thisState;

endcase
```

Figure 7 demonstrates the functionality of the Port module using a test bench. When “portStart” is high, the register write control signal is given to the Port through MUX1, and the ALUB register will be connected the switch data through MUX2.

Listing 3. The case statement is the code for how the Port module determines the direction of the switching of SW8.

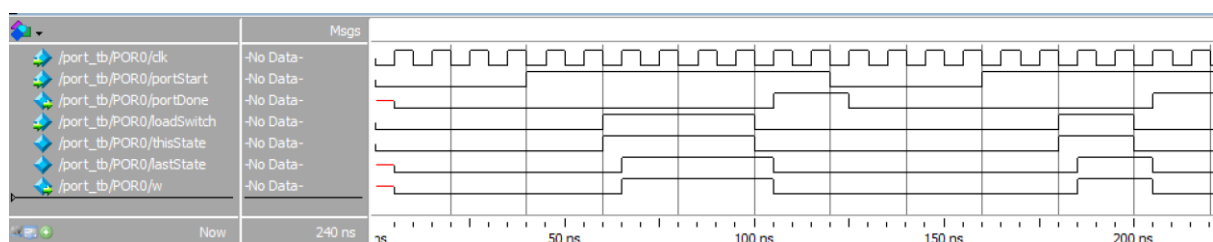


Figure 7. Simulation of Port module. The “w” signal is write and will go high when “loadSwitch” transitions from low to high, and will go low when the opposite happens. “portDone” will go high when “loadSwitch” has done 2 transitions.

4 Data Path

The Data Path consists of two modules, the register module for the GPRs and the ALU (Arithmetic Logic Unit).

4.1 General Purpose Registers

The main differences between my register module and the template given are:

1. Due to my ISA, it is required to have 3 addresses rather than 2, to accommodate for the R type instruction.
2. GPRs at address 2 and 3 will store the results X2 and Y2, and are assigned to “outX2” and “outY2” outputs respectively to display the contents.
3. There is no zero register because there is no need for one, and this without it, it will minimise the logic count.

```
// Declare 4 n-bit registers
logic [n-1:0] gpr [3:0];

// write process to the dstAddr
always_ff @ (posedge clk)
begin
    if (w)
        gpr[dstAddr] ≤ Wdata;
end

assign srcData1 = gpr[srcAddr1[1:0]];
assign srcData2 = gpr[srcAddr2[1:0]];
assign dstData = gpr[dstAddr];
assign outX2 = gpr[2];
assign outY2 = gpr[3];
```

Listing 4. GPR. The synchronous write is seen in the always_ff block. The remaining combinational logic are for reads.

The GPRs instantiate 4 registers, each 8 bits in size, and they serve as the general purpose registers as in listing 4.

The lowest 2 bits of src1 and src2 addresses are used to output the contents of the registers at those addresses. These are done by simple assign statements as shown in listing 4, and thus it is possible to asynchronously read from the registers. As for the “dstData”, this needs to be read as well so that immediate functionality can be included; also implemented by an assign statement.

Writing to a register is asynchronous, as in the template, and since the destination address is 2 bits anyways, there is no need to splice the address as src1 and src2 were.

During development, I realised that I could have assigned 2 registers to always take the switch inputs, and be controlled by a multiplexer selected with SW8. However, this meant that those 2 registers could not be written to by normal means, and must only be read. Because 4 registers is the minimum for the affine transformation, and I’ve already changed my ISA to accommodate for only 2 bit addresses, adding two extra registers would mean that the ISA needed to accommodate for 3 bit addresses. Although this could easily be done, in the spirit of making the system more robust, I would make sure that 3 bit address can fully address all 8 registers. Thus, pushing the total number of registers to 8. Seeing as I could do the affine algorithm with half the registers and a smaller instruction size, I decided against using 2 registers tied to the switch inputs. However, like I have done with the “outX2” and “outY2” signals, these don’t interfere with the register’s ability to read or write, and so they could be used as efficient ways of displaying results.

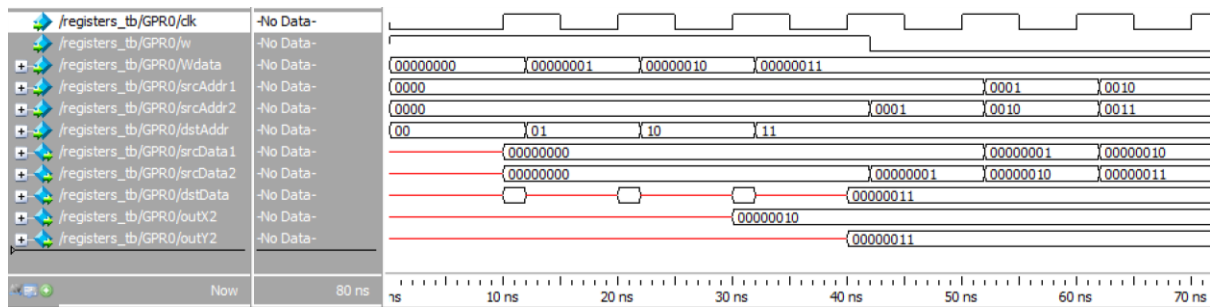


Figure 8. Simulation of register module. Firstly the testbench writes to all 4 registers, then the write “w” signal is switched off, to display the contents of all of the registers as in “srcData1” and “srcData2.”

You can see that in figure 8 the “outX2” and “outY2” signals are unknown until their corresponding registers are written to, and after that, they will always have a value because there is no reset on the GPRs. You can see that the final write to register 3 (displayed by “outY2”), which occurs at about 32ns, but does not actually write to the register until the following clock cycle at 40ns.

4.2 ALU

The ALU is similar to the provided template, however it is significantly smaller due to the lack of subtract hardware and ALU flags. Since, I did not use branch conditions, ALU flags such as the zero flag were seen as redundant and removed. 4 ALU codes were implemented with the pre-processor instruction `DEFINE:

ALUCODE	ASSOCIATED OPCODE	DESCRIPTION
RA	default	Pass the contents of ALU input A to result
RB	LDI, LDP	Pass the contents of ALU input B to result
RADD	ADD, ADDI	Pass the contents of addition result (ar) to result
RMUL	MUL, MULI	Pass the 14 th to 7 th bit of multiplication result (mr) to result

Table 4. ALU codes that instruct the function of the ALU.

Addition and multiplication are done combinatorially by instantiating a signed 8 bit adder and a signed 8 to 16 bit multiplier as shown in the ELEC6234 lectures. The multiplier uses one of the embedded multipliers on the Cyclone V, and it is created as a module as seen in figure 8. The 14th to 7th bit of the 16 bit multiplied result was taken as an 8 bit 2’s complement result.

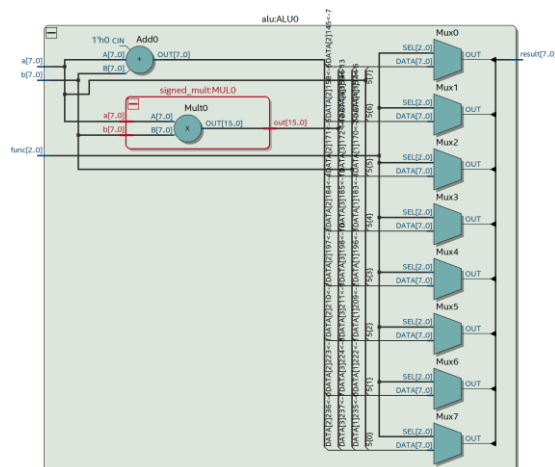


Figure 9. RTL diagram of ALU. You can see that a hardware multiplier and adder were synthesised.

The adder is signed by using an assign statement that takes the result of the addition of the two signed inputs of the ALU. Using the System Verilog keyword “signed” allows the values to be interpreted as 2’s complement, and this was deemed the simplest way of implementing 2’s complement addition and multiplication logic.

The ALU behaves agnostic to what the inputs are, what I mean by this is that it does not know whether the inputs are read from a register or from the switches as this is all handled outside of the ALU through the external multipliers.

/alu_tb/ALU0/a	8	(4	1	8	5	8
/alu_tb/ALU0/b	-68	5	3	-68	30	-68
/alu_tb/ALU0/func	011	000	001	010	011	
/alu_tb/ALU0/result	11111011	00000100	00000101	00000100	11000100	00000001
/alu_tb/ALU0/ar	-60	9	4	-60	35	-60
/alu_tb/ALU0/mr	1111110111100000	0000000000010100	0000000000000011	1111110111100000	0000000010010110	1111110111100000

Figure 10. Simulation of ALU. The inputs and addition result (ar) signals were shown as signed decimal for simpler checking. The result and multiplication result (mr) were kept in binary as I needed to check the 14th to 7th bit of the mr matches the result.

The simulation only tested for a few cases, but there was also another more exhaustive test done on modelsim to check for the correct functionality. Another test was conducted with the affine transformation in mind, and figure 11 demonstrates the functionality with fractions.

/alu_tbAffine/ALU0/a	-No Data-	12	-68
/alu_tbAffine/ALU0/b	-No Data-	-64	96
/alu_tbAffine/ALU0/func	-No Data-	011	
/alu_tbAffine/ALU0/result	-No Data-	-6	-51
/alu_tbAffine/ALU0/ar	-No Data-	-52	28
/alu_tbAffine/ALU0/mr	-No Data-	-768	-6528

Figure 11. Simulation of ALU. Note that “b” (ALUB) is actually the signed decimal representation of the binary number that would be a fraction. -64 is -0.5, and 96 is 0.75. “a” ALUA is not a fraction and so the representation is accurate.

Thus we can see that with inputs 12 and -0.5, the result is -6, which is correct, and the second calculation is also.

4.3 Affine Transformation

As seen in section 2, there are parts of the affine transformation where you must multiply a fraction with a whole number. To do this, the fraction must be represented in 2’s complement binary with a specific weighting on each bit as in section 2. This multiplication will result in a 16 bit number where the weighting are as below.

-2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}
1	1	1	1	1	1	0	1	1	0	0	0	0	0	0	0

Using this weighting, we will take the whole number part of the 16 bit binary, which are bits [14:7]. This will give us the mathematical floor of the fractioned result, as the fractioned part (bits[6:0]) are ignored.

5 FPGA Implementation

With all the individual components tested for functionality as well as the whole system, the next step was to synthesise the HDL on Quartus 16.1. Using the provided “clock.sv” and “picoMIPTest.sv,” the switches, LEDs, fast clock and reset were mapped to the desired FPGA pins with reference to the DE1-SoC manual [2].

On synthesis, there were a number of errors that I did not foresee from simulating. Firstly, since the hex file that was used translates into sets of 4 binary bits for each hex bit, and because my instruction size was 13, the hex file translate to 16 binary bits. This means that the values were truncated and the top 3 bits were ignored. Although this is not an error, it is not ideal and so another option was tried with using the function `$readmemb`, which reads binary files instead. This was not used in the final version however, but is included as an option.

The other main error was that the Port module could not be synthesised due to latches of “thisState”, and this was fixed through using a combinational assign statement as I have shown previously. There were no other significant errors of note, and as many warnings as possible were fixed.

The evaluation method was to trial 5 randomly picked X1 and Y1 values, and check by hand to see if the answer displayed on the LEDs were correct.

X1	Y1	X2	Y2	Correct?
10000000	10000000	10000000	11001100	
11110010	11101000	11111101	11100001	
00000100	11000101	11111001	10111101	
00101000	01010101	01011100	00010111	
00001100	00100001	00101101	11111110	

Table 4. Evaluation matrix. Based on a set of inputs (X1 and Y1), the outputs (X2 and Y2) were recorded.



Figure 15. (left): X2 result for X1=8'b00001100. (right): Y2 result for Y2=8'b00100001.

Figure 15 shows the correct result for the inputs.

6 Conclusion

Testing the design demonstrated that the design does work, although it was tested non-exhaustively, indicating that there is a possibility that it will produce the wrong outputs based on certain inputs. If I had more time, I would write a script in Python or Matlab to calculate all the outputs on every possible input, and trial the FPGA by hand with a sample of 100 of them.

Flow Status	Successful - Thu May 02 11:37:53 2019
Quartus Prime Version	16.1.0 Build 196 10/24/2016 SJ Lite Edition
Revision Name	picoMIPS
Top-level Entity Name	picoMIPS4test
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	66 / 32,070 (< 1 %)
Total registers	64
Total pins	20 / 457 (4 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	1 / 87 (1 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

The synthesis progressed much smoother than anticipated, and this is likely due to the correct use of System Verilog from analysing the template, and also test benching each individual module for functionality.

On inspecting the Quartus Compilation Report on the left, we can see the total number of Adaptive Logic Modules (ALM) is 66, and that there was indeed one embedded multiplier used as there is one DSP block.

We also know that we used 169 bits of RAM.

$$\text{Cost} = 66 \text{ ALMs} + \max(0, 1 \text{ multiplier} - 2) + 30 \times 0.169 \text{ Kbits of RAM} \\ = \mathbf{71.07}$$

In conclusion, the project was relatively successful, however the main improvement I would make is to implement a better Port module which did not rely on the clock speed, or make it use a separate faster clock. I would also prefer to spend more time on the testing of the synthesised design as I felt that my testing was rather shallow. If I had more time to work on the project, it would be wise to write a program in Python or some kind of scripting that could compile the hex files for me. Hand writing the hex files took quite a long time, and this job would suit a program nicely.

7 References

- [1] "Instruction Set Architecture (ISA)", *Cs.kent.edu*, 2019. [Online]. Available: <http://www.cs.kent.edu/~durand/CS0/Notes/Chapter05/isa.html>. [Accessed: 01- May- 2019].
- [2] DE1-Soc User Manual. Altera, 2013 [Online]. Available: http://www.ee.ic.ac.uk/pcheung/teaching/ee2_digital/DE1-SoC_User_manual.pdf. [Accessed: 01- May- 2019].